

Universidad Latina de Costa Rica

Ingeniería en Sistemas Computacionales

Análisis y Diseño de Sistemas I

Proyecto Final: Primera Etapa

Profesor: Lic. Christopher Seas

Estudiantes:

Marcos Vargas Borge

Sebastián José Mora Ortiz

Josué Andrés Rojas Vasquez

Sebastián José Rojas Quiros

Yesica Madriz Chaves

Fecha: 12 de julio de 2026

1. Introducción

En esta segunda etapa del proyecto de Análisis y Diseño de Sistemas II, pasamos de la teoría a la práctica para empezar a darle forma real al sistema de Paradise Resort. Después de entender qué necesita el hotel y hasta dónde queremos llegar, el foco ahora está en definir cómo se va a construir la plataforma por dentro y por fuera.

Para lograrlo, este documento detalla la estructura técnica del software: desde cómo se comunican sus componentes y cómo se va a desplegar, hasta el diseño visual de las pantallas con las que interactúa el usuario. Además, para asegurar que el sistema final sea estable, seguro y fácil de mantener a futuro, definimos los pasos clave para integrar el código, probar que todo funcione sin errores y controlar las distintas versiones del software. Con este plan, nos aseguramos de que el desarrollo sea ordenado y responda exactamente a lo que el hotel necesita para dejar atrás los registros manuales.

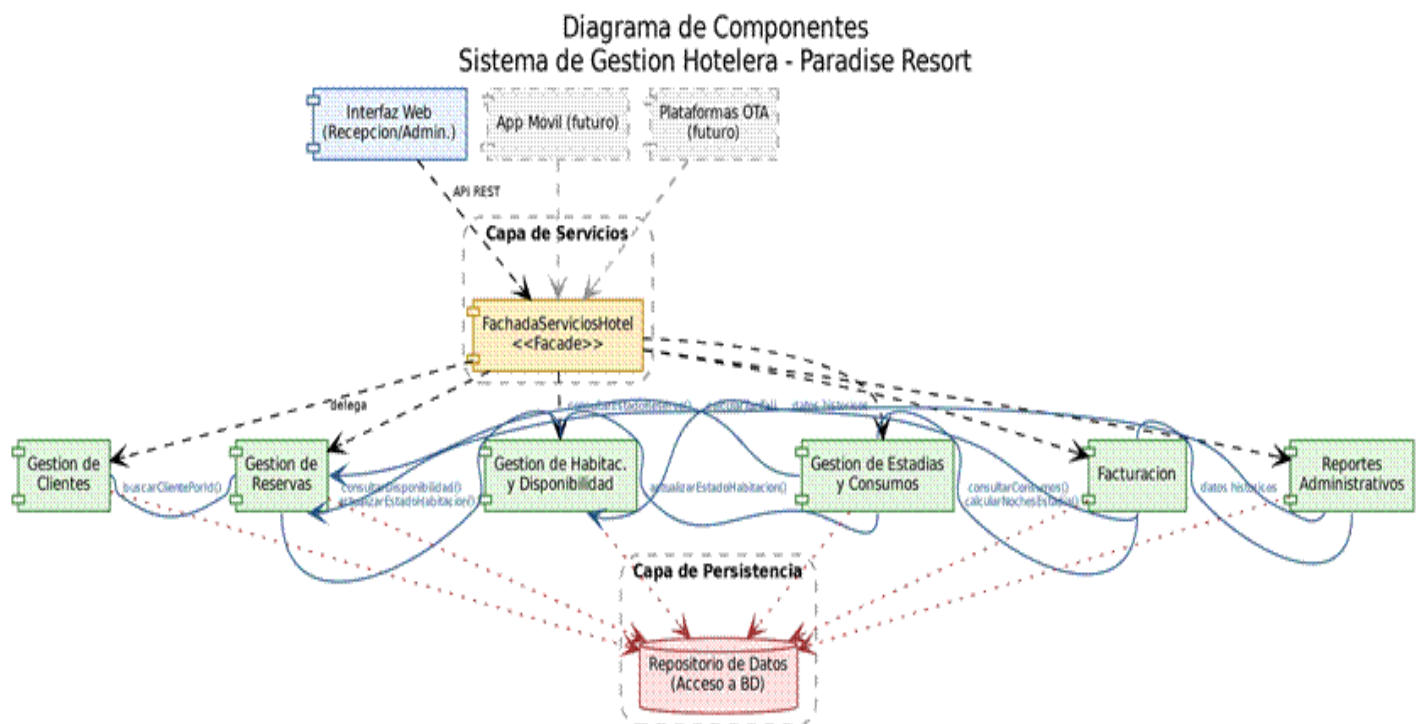
Diagramas de componentes

El siguiente diagrama modela la estructura interna del sistema en términos de componentes de software y sus dependencias, tomando como base la arquitectura en capas definida en la Etapa 1 (Presentación, Servicios, Negocio y Persistencia) y los seis componentes de negocio detallados en el diseño detallado de componentes de esta segunda etapa: Gestión de Clientes, Gestión de Habitaciones y Disponibilidad, Gestión de Reservas, Gestión de Estadías y Consumos, Facturación y Reportes Administrativos.

La capa de Presentación (interfaz web actual, y a futuro app móvil y plataformas OTA) no invoca directamente a los componentes de negocio, sino que consume una única fachada de servicios (FachadaServiciosHotel), aplicando el patrón Facade descrito en la sección de patrones de diseño. Internamente, cada componente de negocio declara su interfaz provista (métodos que ofrece) y su interfaz requerida (métodos que necesita de otro componente), reflejando exactamente las dependencias

documentadas en el diseño detallado: Reservas depende de Habitaciones y de Clientes; Estadías y Consumos depende de Habitaciones y de Reservas; Facturacion depende de Estadías y de Habitaciones; y Reportes Administrativos consume informacion historica de Reservas y de Facturacion. Todos los componentes de negocio acceden, a su vez, a un componente comun de la capa de Persistencia.

Figura 1. Diagrama de componentes del Sistema de Gestion Hotelera Paradise Resort.



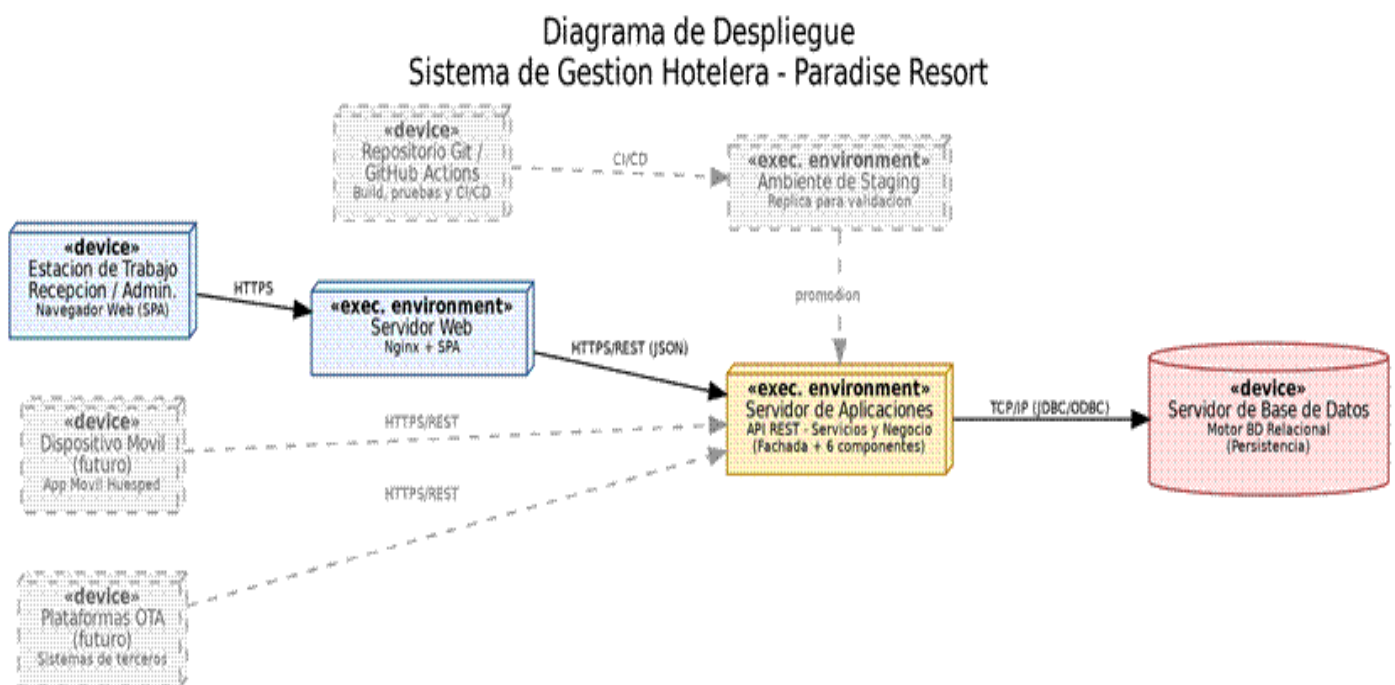
Diagramas de despliegue

El diagrama de despliegue representa la distribucion fisica y logica de los elementos de software sobre los nodos de infraestructura que soportaran el sistema en produccion, en linea con la estrategia de integracion (CI/CD) y con los requerimientos no funcionales de rendimiento (RNF02), concurrencia (RNF03), escalabilidad (RNF04) e integracion (RNF07) establecidos en la Etapa 1.

El personal de recepcion y administracion accede al sistema desde una estacion de trabajo mediante un navegador web (cliente SPA), que se comunica por HTTPS con el Servidor Web. Este ultimo entrega la interfaz de presentacion y reenvia las peticiones, a traves de una API REST, al Servidor de Aplicaciones, donde se ejecutan la Capa de Servicios (FachadaServiciosHotel) y la Capa de Negocio con sus seis componentes.

El Servidor de Aplicaciones se comunica con el Servidor de Base de Datos mediante TCP/IP (JDBC/ODBC) para persistir y consultar la informacion. Adicionalmente, se representa el ambiente de Staging, donde el pipeline de integracion continua (Repositorio Git / GitHub Actions descrito en la estrategia de integracion) despliega automaticamente cada version candidata antes de promoverla a produccion. Los nodos en gris discontinuo (dispositivo movil y plataformas OTA) corresponden a integraciones futuras previstas por la arquitectura, pero no forman parte del alcance actual del proyecto.

Figura 2. Diagrama de despliegue del Sistema de Gestion Hotelera Paradise Resort.



Diseño de interfaces

El diseño de las pantallas se pensó alrededor de las tareas que más se repiten en el día a día del hotel: crear una reserva, hacer check-in/check-out y cerrar la cuenta de un huésped. Por eso se aplicaron los siguientes principios de diseño centrado en el usuario:

- **Consistencia:** todas las pantallas comparten el mismo menú lateral y la misma barra superior, para que el recepcionista no tenga que reaprender la navegación al cambiar de módulo.
- **Visibilidad del estado del sistema:** el panel principal muestra de un vistazo el estado de todas las habitaciones (disponible, ocupada, reservada, en mantenimiento) con colores diferenciados, atacando directamente el problema de falta de visibilidad de disponibilidad descrito en la problemática.
- **Prevención de errores:** antes de confirmar una reserva, el sistema verifica la disponibilidad y muestra un aviso visual (RF09), evitando que se genere una reserva duplicada sobre la misma habitación.
- **Retroalimentación inmediata:** acciones como registrar un check-in, aplicar un descuento o emitir una factura muestran una confirmación visual clara antes de continuar.
- **Minimizar la carga de memoria del usuario:** en check-in/check-out y en facturación, los datos del huésped y de la reserva se mantienen visibles en la parte superior de la pantalla en todo momento, sin obligar al recepcionista a buscarlos de nuevo.
- **Jerarquía visual clara:** las tarjetas de resumen (KPIs) y las tablas separan la información numérica de las acciones disponibles, y los botones principales como “Confirmar reserva” o “Emitir factura” resaltan sobre las acciones secundarias.

Diseño de interfaces

El diseño de las pantallas se pensó alrededor de las tareas que más se repiten en el día a día del hotel: crear una reserva, hacer check-in/check-out y cerrar la cuenta de un

huésped. Por eso se aplicaron los siguientes principios de diseño centrado en el usuario:

- Consistencia: todas las pantallas comparten el mismo menú lateral y la misma barra superior, para que el recepcionista no tenga que reaprender la navegación al cambiar de módulo.
- Visibilidad del estado del sistema: el panel principal muestra de un vistazo el estado de todas las habitaciones (disponible, ocupada, reservada, en mantenimiento) con colores diferenciados, atacando directamente el problema de falta de visibilidad de disponibilidad descrito en la problemática.
- Prevención de errores: antes de confirmar una reserva, el sistema verifica la disponibilidad y muestra un aviso visual (RF09), evitando que se genere una reserva duplicada sobre la misma habitación.
- Retroalimentación inmediata: acciones como registrar un check-in, aplicar un descuento o emitir una factura muestran una confirmación visual clara antes de continuar.
- Minimizar la carga de memoria del usuario: en check-in/check-out y en facturación, los datos del huésped y de la reserva se mantienen visibles en la parte superior de la pantalla en todo momento, sin obligar al recepcionista a buscarlos de nuevo.
- Jerarquía visual clara: las tarjetas de resumen (KPIs) y las tablas separan la información numérica de las acciones disponibles, y los botones principales como “Confirmar reserva” o “Emitir factura” resaltan sobre las acciones secundarias.

Estas pantallas se diseñaron en baja fidelidad (wireframes en escala de grises), ya que en esta etapa el objetivo es validar la estructura, el contenido y el flujo de cada pantalla, y no el estilo visual final del producto, que se definirá durante la construcción.

Wireframes o mockups

A continuación se presentan los wireframes de las siete pantallas principales del sistema, elaborados en baja fidelidad para representar la estructura y el flujo de cada proceso descrito en los requerimientos funcionales.

PARADISE RESORT
Sistema de Gestión Hotelera

Usuario
recepcion01

Contraseña

☐ Recordarme [¿Olvidó su contraseña?](#)

INICIAR SESIÓN

Acceso restringido a personal autorizado — Recepción / Administración

Figura 1. Inicio de sesión. Acceso diferenciado para recepción y administración.

Panel Principal — Disponibilidad de Habitaciones Recepcionista: M. Vargas | 13/07/2026

☒ Panel principal

- ☐ Reservas
- ☐ Check-in / Check-out
- ☐ Consumos
- ☐ Facturación
- ☐ Reportes
- ☐ Clientes

42 / 60
Habitaciones ocupadas

11
Disponibles

7
Reservadas hoy

3
En mantenimiento

Tipo de habitación: Todos

Fecha entrada: 13/07/2026

Fecha salida: 15/07/2026

+ NUEVA RESERVA

Estado de habitaciones

101 disponible	102 ocupada	103 ocupada	104 reservada	105 disponible	106 mantenimiento	107 ocupada	108 disponible
201 ocupada	202 disponible	203 reservada	204 ocupada	205 ocupada	206 disponible	207 ocupada	208 reservada

■ Disponible ■ Reservada ■ Ocupada ■ Mantenimiento

Figura 2. Panel principal. Vista consolidada de disponibilidad de habitaciones por estado (RF05-RF07).

Nueva Reserva

Recepcionista: M. Vargas | 13/07/2026

☐ Panel principal

☒ Reservas

☐ Check-in / Check-out

☐ Consumos

☐ Facturación

☐ Reportes

☐ Clientes

1. Datos del cliente

Cliente (buscar por identificación)

1-0234-0567

BUSCAR / REGISTRAR NUEVO

Nombre completo

Ana Pérez Ramírez

Teléfono / Correo

8888-2211

2. Detalles de la reserva

Fecha de entrada

20/07/2026

Fecha de salida

23/07/2026

Número de huéspedes

2

Tipo de habitación

Doble – Vista al mar

Canal de origen

Teléfono

☐ Disponibilidad verificada: habitación 204 disponible para las fechas seleccionadas

3. Servicios adicionales (opcional)

☐ Desayuno bufé

☐ Transporte aeropuerto

☐ Lavandería

☐ Actividad recreativa

Tarifa preliminar: 3 noches x \$85 = \$255.00

Estado de la reserva: Pendiente de confirmación

CANCELAR

CONFIRMAR RESERVA

Figura 3. Nueva reserva. Verificación automática de disponibilidad antes de confirmar (RF08-RF12).

Check-in / Check-out — Habitación 204

Recepcionista: M. Vargas | 13/07/2026

☐ Panel principal

☐ Reservas

☒ Check-in / Check-out

☐ Consumos

☐ Facturación

☐ Reportes

☐ Clientes

Buscar por número de habitación o cliente

204

BUSCAR

Huésped: Ana Pérez Ramírez

Reserva #00123 — Doble Vista al Mar — 20/07/2026 al 23/07/2026 (3 noches)

Estado actual de la reserva: Confirmada

RESERVADA

CHECK-IN

CHECK-OUT

Fecha y hora de check-in

20/07/2026 14:05

Número de huéspedes adicionales

1

Observaciones

Solicita cuna adicional

REGISTRAR CHECK-IN

Al confirmar, la habitación 204 pasa a estado "Ocupada".

Consumos registrados en esta estadía

Servicio	Descripción	Fecha	Monto
Restaurante	Cena – mesa 4	20/07 20:15	\$28.50
Lavandería	2 prendas	21/07 09:40	\$6.00
Transporte	Aeropuerto – hotel	20/07 13:00	\$18.00

Figura 4. Check-in / check-out. Registro de entrada, salida y consumos asociados a la estancia (RF13-RF20).

Facturación — Check-out Habitación 204

Recepcionista: M. Vargas | 13/07/2026

☐ Panel principal

☐ Reservas

☐ Check-in / Check-out

☐ Consumos

☒ Facturación

☐ Reportes

☐ Clientes

Huésped: Ana Pérez Ramírez

Reserva #00123 — Habitación 204 — 20/07/2026 al 23/07/2026 (3 noches)

Detalle de hospedaje

Concepto	Cantidad	Precio unitario	Subtotal
Habitación Doble – Vista al mar	3 noches	\$85.00	\$255.00

Consumos adicionales

Servicio	Descripción	Fecha	Monto
Restaurante	Cena – mesa 4	20/07/2026	\$28.50
Lavandería	2 prendas	21/07/2026	\$6.00
Transporte	Aeropuerto – hotel	20/07/2026	\$18.00

Método de pago

Tarjeta de crédito

Estado del pago

Pagado

Subtotal hospedaje

\$255.00

Subtotal consumos

\$52.50

Descuento aplicado

-\$10.00

TOTAL A PAGAR

\$297.50

REIMPRIMIR

EMITIR FACTURA

Figura 5. Facturación. Consolidación de hospedaje y consumos en la factura final (RF21-RF24).

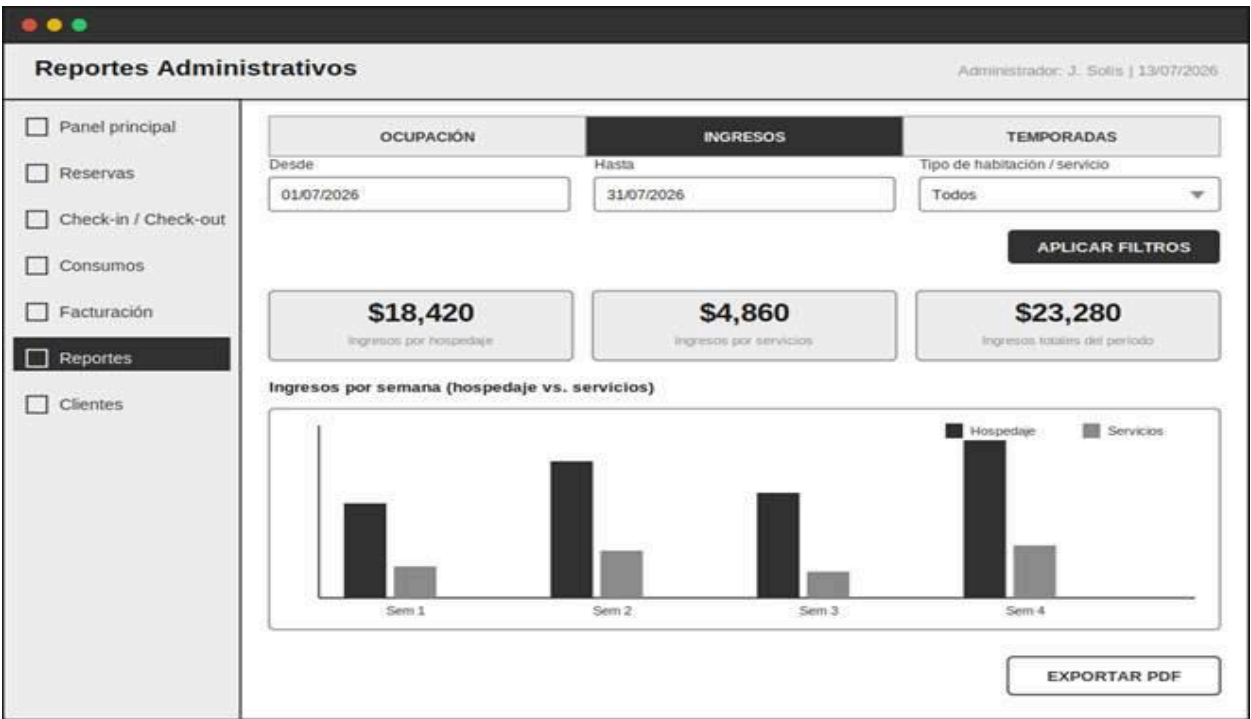


Figura 6. Reportes administrativos. Filtros por período e indicadores de ocupación e ingresos (RF25-RF28).

Panel de Gestión de Clientes Recepcionista: M. Vargas | 13/07/2026

☐ Panel principal
☐ Reservas
☐ Check-in / Check-out
☐ Consumos
☐ Facturación
☐ Reportes
☒ Clientes
☐ Reservas

Clientes Registrados

ID	Nombre	Teléfono/Correo	Reservas
1-0234-0567	Ana Pérez Ramírez	8888-2211	3
2-0345-0678	Carlos López González	6012-3456	1
3-0345-0679	Maria Fernandez Solis	maria.f@mail.com	5
4-0235-0566	Maria Conzález	maria.f@mail.com	5
5-0345-0677	Carlos González	6012-3456	1
2-0345-0678	Maria Mernorez-Risovo	maria.f@mail.com	4
7-0345-0678	Carlos López	maia.f@mail.com	1
3-0456-0789	Maria Fernández Solís	maria.f@mail.com	5

1-10 de 123 | < >

NUEVO CLIENTE

Aplicación de patrones de diseño

Para el diseño interno del sistema se eligieron seis patrones del catálogo GoF, cada uno pensado para resolver un problema puntual que salió de los requerimientos de la Etapa 1. La idea no fue meter patrones por meterlos, sino usarlos donde de verdad simplifican el diseño:

- Facade – Fachada de servicios: en vez de que la interfaz web, la futura app móvil y las plataformas externas de reservación tengan que hablarle por separado a cada uno de los seis componentes del sistema, se crea una FachadaServiciosHotel que centraliza operaciones como crearReserva() o generarFacturaCheckOut() y internamente delega el trabajo al componente correspondiente. Responde a RNF09 (integración futura) y RNF06 (mantenibilidad).

- Strategy – Cálculo de tarifas: el monto de hospedaje (RF16) puede variar según el tipo de habitación y, a futuro, según temporada o promociones. Por eso se usa una interfaz `EstrategiaTarifa` con implementaciones como `TarifaEstandar` o `TarifaTemporadaAlta`, de modo que agregar una tarifa nueva no obliga a tocar el componente de Facturación (RF04, RNF06).
- State – Estados de habitación y reserva: una habitación (disponible, ocupada, reservada, en limpieza, en mantenimiento) y una reserva (pendiente, confirmada, cancelada, completada) se comportan distinto según su estado (RF06, RF07, RF10). En lugar de llenar el código de condicionales, cada estado es una clase que sabe a qué otros estados puede pasar, lo cual también ayuda a evitar inconsistencias cuando varios usuarios trabajan al mismo tiempo (RNF04).
- Decorator – Consumos adicionales: durante la estadía se van sumando consumos de restaurante, lavandería, transporte o actividades (RF17-RF20). Cada consumo se modela como un decorador que envuelve la cuenta base y le suma su monto, sin que la clase `Estadia` tenga que conocer de antemano todas las combinaciones posibles de servicios.
- Builder – Construcción de la factura: la factura final junta hospedaje, consumos, descuentos y método de pago (RF21-RF23). Un `FacturaBuilder` arma el objeto por pasos (`conMontoHospedaje()`, `conConsumos()`, `conDescuento()`) hasta ensamblar la Factura completa al llamar `construir()`, lo que evita que ese armado quede regado en varias partes del código.
- Template Method – Generación de reportes: los reportes de ocupación, ingresos y temporada (RF25-RF28) siguen el mismo esqueleto —traer datos, filtrar, calcular, dar formato— pero cada uno calcula algo distinto. Una clase `ReporteBase` fija ese orden general y cada subclase (`ReporteOcupacion`, `ReporteIngresos`, `ReporteTemporada`) solo implementa su parte específica.

En conjunto, estos patrones mantienen el sistema modular y con bajo acoplamiento entre componentes, algo clave para que el equipo pueda seguir trabajando sobre el diseño detallado de la siguiente sección sin fricciones.

Diseño detallado de componentes

A partir de la arquitectura en capas definida en la Etapa 1 (sección de Arquitectura Preliminar), se detalla a continuación el diseño interno de cada uno de los seis componentes de negocio identificados en la capa de aplicación, especificando su responsabilidad principal, las clases que lo conforman, la interfaz que provee hacia la capa de Servicios/API, la interfaz que requiere de otros componentes, y los patrones de diseño aplicados internamente (descritos en la sección anterior).

- **Componente: Gestión de Clientes:** administra el ciclo de vida de la información de los huéspedes.
 - Responsabilidad: registrar, actualizar, consultar y mantener el historial de clientes/huéspedes, conforme a RF01-RF03.
 - Clases internas principales: Cliente, HistorialCliente.
 - Interfaz provista: registrarCliente(datos), actualizarCliente(id, datos), consultarHistorial(idCliente), buscarClientePorIdentificacion(identificacion).
 - Interfaz requerida: ninguna dependencia directa de otros componentes de negocio; consume únicamente la capa de datos.
 - Patrones aplicados: ninguno de forma exclusiva; sus clases participan como datos de contexto en los patrones Builder y Decorator de otros componentes (por ejemplo, el cliente asociado a una reserva o factura).

- **Componente: Gestión de Habitaciones y Disponibilidad:** administra el catálogo de habitaciones, sus tipos, tarifas y estado en tiempo real.
 - Responsabilidad: administrar el catálogo de habitaciones, sus tipos y tarifas, así como el estado y disponibilidad en tiempo real, conforme a RF04-RF07.
 - Clases internas principales: Habitacion, TipoHabitacion, EstrategiaTarifa (y sus implementaciones concretas), EstadoHabitacion (y sus estados concretos).

- Interfaz provista: consultarDisponibilidad(tipoHabitacion, fechaEntrada, fechaSalida), actualizarEstadoHabitacion(idHabitacion, nuevoEstado), calcularTarifa(idHabitacion, noches).
 - Interfaz requerida: ninguna dependencia directa de otros componentes de negocio.
 - Patrones aplicados: Strategy (cálculo de tarifas) y State (transiciones de estado de la habitación).
- Componente: Gestión de Reservas: administra el ciclo completo de la reserva, centralizando la verificación de disponibilidad.
 - Responsabilidad: administrar la creación, confirmación, modificación y cancelación de reservas, centralizando la verificación de disponibilidad para evitar duplicidades, conforme a RF08-RF12.
 - Clases internas principales: Reserva, EstadoReserva (y sus estados concretos).
 - Interfaz provista: crearReserva(datosReserva), modificarReserva(idReserva, datos), cancelarReserva(idReserva), consultarEstadoReserva(idReserva).
 - Interfaz requerida: consultarDisponibilidad() y actualizarEstadoHabitacion() del componente de Gestión de Habitaciones y Disponibilidad; buscarClientePorIdentificacion() del componente de Gestión de Clientes.
 - Patrones aplicados: State (transiciones de estado de la reserva); Facade (expone sus operaciones a través de la fachada de servicios junto con los demás componentes).
- Componente: Gestión de Estadías y Consumos: administra el check-in, el check-out y la acumulación de consumos adicionales.

- Responsabilidad: administrar el proceso de check-in y check-out, así como el registro y acumulación de consumos adicionales durante la estadía, conforme a RF13-RF20.
 - Clases internas principales: Estadia, ComponenteCuenta, CuentaEstadia, decoradores concretos de consumo (ConsumoRestaurante, ConsumoLavanderia, ConsumoTransporte, ConsumoActividad), ServicioAdicional.
 - Interfaz provista: registrarCheckIn(idReserva), registrarCheckOut(idEstadia), registrarConsumo(idEstadia, tipoServicio, monto), consultarConsumosAcumulados(idEstadia).
 - Interfaz requerida: actualizarEstadoHabitacion() del componente de Gestión de Habitaciones y Disponibilidad; consultarEstadoReserva() del componente de Gestión de Reservas.
 - Patrones aplicados: Decorator (acumulación de consumos adicionales sobre la cuenta de la estadía).
- Componente: Facturación: consolida el monto de hospedaje y los consumos adicionales en la factura final.
 - Responsabilidad: consolidar el monto de hospedaje y los consumos adicionales de la estadía en una factura final, aplicar descuentos y registrar el estado del pago, conforme a RF21-RF24.
 - Clases internas principales: Factura, FacturaBuilder.
 - Interfaz provista: generarFactura(idEstadia), aplicarDescuento(idFactura, monto, justificacion), registrarPago(idFactura, metodoPago), consultarFactura(idFactura).
 - Interfaz requerida: consultarConsumosAcumulados() y calcularNochesEstadia() del componente de Gestión de Estadías y Consumos; calcularTarifa() del componente de Gestión de Habitaciones y Disponibilidad.
 - Patrones aplicados: Builder (ensamblaje paso a paso de la factura consolidada).

- **Componente: Reportes Administrativos:** genera los reportes gerenciales de ocupación, ingresos y temporadas de mayor demanda.
 - **Responsabilidad:** generar reportes de ocupación, ingresos y temporadas de mayor demanda a partir de la información histórica del sistema, conforme a RF25-RF28.
 - **Clases internas principales:** ReporteBase (clase plantilla), ReporteOcupacion, ReporteIngresos, ReporteTemporada.
 - **Interfaz provista:** generarReporteOcupacion(rangoFechas), generarReporteIngresos(rangoFechas), generarReporteTemporada(rangoFechas).
 - **Interfaz requerida:** información histórica de los componentes de Gestión de Reservas, Gestión de Habitaciones y Disponibilidad, y Facturación, consultada a través de sus respectivas interfaces provistas.
 - **Patrones aplicados:** Template Method (esqueleto común de generación de reportes).

Esta descomposición interna respeta los principios de modularidad, alta cohesión y bajo acoplamiento establecidos en la Etapa 1: cada componente expone una interfaz provista reducida y coherente, declara explícitamente sus interfaces requeridas de otros componentes, y aplica internamente los patrones de diseño que resuelven de forma más adecuada la problemática específica de su dominio.

Estrategia de integración

Para asegurarnos de que todas las partes del sistema se acoplen de forma ordenada y sin interrupciones, vamos a usar una estrategia de integración y despliegue continuo (CI/CD) basada en los siguientes puntos:

- **Enfoque de integración (Top-Down / Sándwich):** Básicamente, vamos a priorizar la integración de los servicios del núcleo o backend usando mockups de las pantallas en las primeras etapas. Una vez que las API y la base de datos estén sólidas, iremos metiendo las interfaces reales poco a poco.

- **Automatización con Git y GitHub Actions:** Cada vez que alguien termine una funcionalidad o arregle un fallo, tendrá que abrir un Pull Request hacia la rama de desarrollo. Esto disparará automáticamente un flujo de trabajo que:
 - Compila el código para verificar que no haya errores de sintaxis o de dependencias.
 - Corre las pruebas unitarias automáticas.
 - Revisa la calidad del código con herramientas de análisis estático (linter).
- **Ambientes de Staging:** Ningún cambio va a ir directo a producción. Tendremos un entorno de pruebas idéntico al real (Staging) para desplegar las versiones candidatas y validar que la comunicación entre módulos, bases de datos y servicios externos funcione bien antes del lanzamiento final.

Plan de pruebas

Este plan es el que nos garantiza que el sistema sea estable, seguro y rinda bien antes de que nadie más lo use. Lo organizamos en tres niveles:

1. Tipos de pruebas a ejecutar

- **Pruebas unitarias (Unit Tests):** Sirven para revisar el comportamiento aislado de funciones específicas, controladores y la lógica del negocio. Nos vamos a exigir una cobertura mínima del 80% del código fuente.
- **Pruebas de integración:** Diseñadas para chequear que la interacción entre los diferentes módulos (por ejemplo, que la API hable bien con la base de datos) funcione como debe y no se rompa la información en el camino.
- **Pruebas de interfaz de usuario (UI/UAT):** Validaciones funcionales basadas en los wireframes y diseños de pantalla para asegurar que el usuario final pueda hacer sus tareas del día a día sin enredos.

2. Entorno y criterios de aceptación

- Criterios de éxito: Una versión estará lista para desplegarse si pasa el 100% de las pruebas críticas de integración, no tiene fallos de seguridad graves en el análisis estático y cumple con los tiempos de respuesta que pide el hotel.
- Reporte de defectos: Cualquier fallo que encontremos se registrará en la herramienta de gestión del proyecto, ordenándose por severidad (Bloqueante, Mayor, Menor, Trivial) para que el equipo de desarrollo sepa qué arreglar primero.

Administración de configuración

Para llevar un control estricto de los cambios en el software y sus versiones, nos organizaremos así:

- **Estrategia de ramificación (GitFlow):** Usaremos un repositorio Git estructurado con las siguientes ramas:
 - o main / master: Código de producción que ya es 100% estable.
 - o develop: La rama principal donde vamos juntando todo lo que se va desarrollando.
 - o feature/*: Ramas temporales para trabajar en tareas específicas, que se borran después de unirse al desarrollo.
 - o hotfix/*: Ramas de emergencia para arreglar fallos críticos directos en producción.
- **Gestión de dependencias y variables de entorno:** Todas las librerías del sistema se controlarán con los archivos de configuración de cada lenguaje (como package.json, pom.xml o requirements.txt) para que el proyecto corra igual en la máquina de cualquiera. Además, las contraseñas, llaves de API y conexiones a la base de datos se manejan solo en archivos .env ocultos o gestores de secretos; queda totalmente prohibido dejarlas escritas directamente en el código.